QFabric: Contract-Driven Recovery Across a Linux–RTOS Boundary

Ayushman Bhattacharya
Pollinations AI
ayushman.elixpo@gmail.com

Anwesha Chakraborty
Department of Computer Science and Engineering
JIS University
anwesha.elixpo@gmail.com

Prepublication manuscript. Copyright © 2026 Ayushman Bhattacharya and Anwesha Chakraborty. All Rights Reserved.

***Abstract*—Heterogeneous embedded boards increasingly combine an application processor running Linux with a microcontroller running a real-time operating system. Choosing one domain statically is simple, but timing behavior can change after deployment, while unconstrained migration can violate program semantics or overload the real-time domain. We present QFabric, a prototype that treats placement as a closed-loop recovery problem for empirical soft real-time contracts. It profiles the same bounded task in both domains, observes end-to-end latency windows, rejects semantically or temporally inadmissible moves, switches only at a quiescent epoch boundary, and commits or rolls back after a probation window. On an Arduino UNO Q, a Linux–Zephyr platform, direct measurements characterize communication, profiling, decision, and instrumentation overheads. Deterministic replay of those measured traces over four application graphs compares seven policies. In a four-node load-shift case, ordinary Linux misses 67.3% of 80-ms deadlines at a 111.30-ms p95; QFabric recovery reduces p95 by 36.6%to 70.61 ms and the miss rate to 3.7%. A tuned `SCHED_FIFO` baseline remains substantially better at 23.91 ms and zero misses. Thus, the result is not that adaptation dominates expert tuning, but that bounded, semantics-aware recovery can salvage an untuned deployment when its original contract stops holding. The complete source, raw evidence, audits, and manuscript generators are released as a reproducible artifact.**

***Index Terms*—heterogeneous embedded systems, Linux, RTOS, runtime adaptation, timing contracts, task migration, empirical evaluation**

I. Introduction

Modern embedded platforms often put two distinct execution environments on one board: a feature-rich Linux application processor and a resource-constrained real-time microcontroller. Linux offers libraries, networking, storage, and developer velocity; the microcontroller offers isolation and more predictable execution. A task that can execute in either domain therefore faces a placement choice whose consequences extend beyond compute time: crossing the boundary adds serialization and transport cost, contention changes tail latency, and a move may be illegal for a stateful or Linux-only operation.

Classical heterogeneous schedulers optimize a static directed acyclic graph from cost estimates [1]. Real-time theory provides strong admission and schedulability foundations [2], while feedback scheduling adapts resources to measured error [3]. Runtime verification and timing contracts make violations observable [4], [5]. Yet an engineering gap remains on asymmetric Linux–RTOS boards: how should a deployed task recover when an empirical timing contract fails, without treating a faster prediction as sufficient permission to migrate?

QFabric addresses that gap as a deliberately narrow systems contribution. A portable QTask declaration is compiled for both domains. Offline measurements seed placement and an empirical contract. At runtime, a monitor applies hysteresis to complete observation windows. A recovery controller admits a candidate only if its semantics, predicted end-to-end cost, real-time utilization, evidence, and cooldown constraints all pass. It then drains in-flight work, increments an epoch, switches, and evaluates a probation window. The move is committed only if post-switch observations satisfy the contract; otherwise it is rolled back.

This paper makes four contributions:

- a cross-domain QTask interface with an explicitly bounded wire ABI and effect annotations used as migration constraints;
- a closed-loop recovery protocol combining empirical timing contracts, RT admission, quiescent epoch switching, and post-switch verification;
- an evidence chain that distinguishes direct hardware measurements from deterministic application-graph trace replay and machine-checks all reported claims; and
- an evaluation against ordinary Linux, tuned Linux, static and manual placements, a non-adaptive QFabric policy, recovery, and an offline oracle.

The central result is intentionally qualified. QFabric improves a failing ordinary-Linux deployment, but it does not outperform correctly tuned Linux or the oracle in our flagship case. This negative comparison clarifies where the prototype is useful: recovery when prior deployment assumptions no longer hold, not replacement of platform-specific real-time engineering.

II. Platform and Problem

A. Target platform

The Arduino UNO Q combines a quad-core Arm Cortex-A53 application processor running Debian Linux with an STM32U585 Cortex-M33 microcontroller running Zephyr [6]. Arduino's Bridge stack exposes MessagePack remote procedure calls in both directions [7]. Its underlying architecture is representative of remote-processor systems built around `remoteproc` and `rpmsg` abstractions [8], [9], but our

implementation targets the stock UNO Q software stack rather than modifying the transport.

B. System model

An application is a directed graph $G = (V, E)$. Each task $t \in V$ may have a Linux implementation, an RT implementation, or both. Its declaration includes fixed-width request and response types, a numeric task identifier, and an effect class. In this prototype, only pure, bounded tasks are migration eligible. Each edge $e \in E$ incurs a measured boundary cost b_e when its endpoints occupy different domains.

For task t in domain d, profiling yields a latency distribution and its empirical 95th percentile $q_{.95}(t, d)$. Given neighboring placements, the policy estimates

$$\widehat{C}(t, d) = q_{.95}(t, d) + \sum_{e \in E_t} \mathbf{1}[d \neq d_{\text{neighbor}(e)}] b_e. \quad (1)$$

This is a transparent heuristic, not a worst-case execution-time proof. The contract deadline D and tolerated miss rate ϵ likewise describe an empirical soft real-time objective.

For a complete window w of n observations with outcome o_i and latency L_i, the observed miss rate is

$$\widehat{m}_w = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}[o_i \neq \text{ok} \ \vee \ L_i > D]. \quad (2)$$

A violation means $\widehat{m}_w > \epsilon$. Consecutive-window thresholds avoid reacting to a single outlier.

C. Research questions

The evaluation asks: **RQ1**, can the controller detect a sustained contract failure without false switches? **RQ2**, can a feasible switch improve the violated contract? **RQ3**, are unsafe or unsupported switches rejected? **RQ4**, how does recovery compare with deployable baselines and an oracle? **RQ5**, what overheads does the mechanism introduce? and **RQ6**, what programmer effort is visible in the prototype?

III. Related Work

HEFT ranks a task graph and maps tasks to heterogeneous processors using estimated computation and communication costs [1]. It is a natural static-placement reference, but it does not monitor deployed timing contracts. Feedback real-time scheduling instead adjusts resource allocation from runtime error [3]; autonomic computing generalizes the monitor–analyze–plan–execute loop [10]. QFabric uses a feedback loop, but its actuator is a guarded cross-domain implementation switch.

Runtime verification can observe real-time behavior with bounded intrusion [4]. Stochastic contracts provide empirical runtime checks for component systems [5], while formal timing contracts and zone-based models support verification and synthesis [11], [12]. These works motivate contract semantics; our focus is the operational recovery protocol after a violation.

Real-time task migration has been studied for dynamic resource management in many-core systems with feasibility analysis [13]. Linux also supplies deadline-aware scheduling mechanisms [14]. QFabric does not replace either: it combines a simple RT utilization guard with empirical cross-domain evidence, and explicitly compares against tuned Linux scheduling.

CoMOS provides processor-agnostic tasks and runtime migration across heterogeneous processors [15], while YASMIN supports functionally equivalent task implementations and user-space real-time mapping on heterogeneous COTS platforms [16]. Awkernel more recently brings DAG scheduling into an RTOS and evaluates it on physical hardware [17]. These systems establish closely related programming and scheduling foundations. QFabric instead focuses on recovery after an observed empirical-contract violation across an already deployed Linux–RTOS boundary, with semantic admission, a quiescent epoch change, and a probation-based commit or rollback.

Table I locates the contribution. A checkmark means the feature is central to the cited work, not that it could not be added. To the best of our literature search, the combined column pattern—empirical violation, semantic guard, RT admission, atomic switch, and post-switch verification on a physical Linux–RTOS boundary—is the distinctive part of QFabric. We do not claim novelty for heterogeneous placement, profiling, feedback control, or timing contracts individually.

IV. QFabric Design

A. QTask declaration and bounded ABI

A QTask declaration is the single source for Linux and MCU artifacts. The generator accepts fixed-size scalars, records, and bounded arrays; it rejects pointers, dynamic arrays, recursive types, unsupported effects, and messages larger than the 988-byte payload budget. Versioned big-endian headers carry a magic value, message kind, task identifier, request identifier, payload length, and flags. Golden vectors are checked independently in Python, C++, and on the MCU. These restrictions turn serialization bounds and effect eligibility into auditable facts rather than conventions.

The current prototype exposes one concrete task, integer addition. That task is purposefully small: it isolates placement and communication behavior, but it also limits external validity. Section VIII-A treats this directly.

B. Contracts and admission

The profiler measures both implementations using warm-up exclusion and retains raw observations. A minimum sample count marks an estimator valid; cold-start runs remain labeled insufficient rather than silently promoted to evidence. The prototype derives its per-node deadline before application replay:

$$D = \left\lceil 1.25 \max_r p99_r \right\rceil_{5\,\text{ms}} = 20\,\text{ms}, \quad (3)$$

where $p99_r$ are three idle round-trip repetitions. The derivation is recorded as non-post-hoc in the artifact.

A candidate RT move must leave utilization beneath a reserved headroom h:

$$U'_{RT} = U_{RT} + r_t \, q_{.95}(t, RT) \leq 1 - h, \quad (4)$$

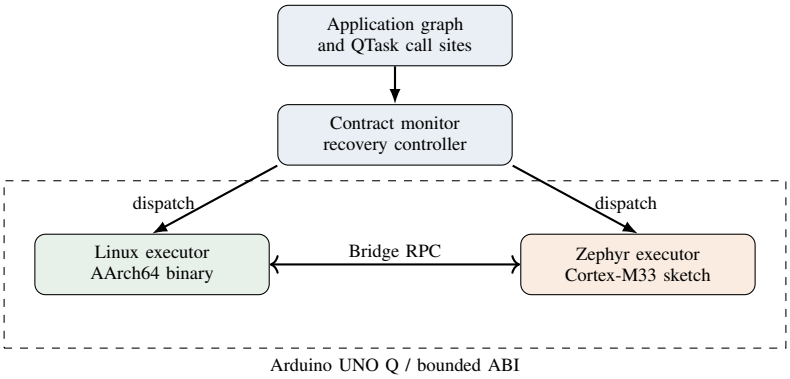


Fig. 1. QFabric architecture. The policy runs in Linux userspace; task code has a native implementation in each domain.

TABLE I
QUALITATIVE POSITIONING AGAINST REPRESENTATIVE PRIOR WORK

Work	Heterogeneous placement	Empirical monitor	Semantic guard	RT admission	Verified switch	Linux–RTOS board
HEFT [1]	✓	–	–	–	–	–
Feedback scheduling [3]	–	✓	–	model-specific	–	–
Stochastic contracts [5]	–	✓	–	–	–	–
RT task migration [13]	✓	–	–	✓	migration protocol	–
CoMOS [15]	✓	–	–	–	migration protocol	–
YASMIN [16]	✓	–	–	–	–	–
Awkernel [17]	✓	✓	–	–	–	–
Bridge / rpmsg [7], [9]	transport only	–	–	–	–	✓
QFabric	✓	✓	✓	✓	✓	✓

where r_t is the invocation rate. A switch is admissible only when

$$A(t,d) = E_{\min} \wedge S_{\text{semantic}} \wedge S_{\text{admission}} \wedge (\widehat{C}(t,d) \leq D) \wedge S_{\text{cooldown}}. \quad (5)$$

The controller reports a stable rejection code for every failed conjunct.

C. Safe recovery protocol

Figure 2 shows the recovery state machine. Three consecutive violating windows trigger analysis. If no candidate is admissible, placement is unchanged. Otherwise the dispatcher stops accepting new work, waits until the in-flight count is zero, increments a routing epoch, and changes the active domain. Six probation windows then provide post-switch evidence. Satisfaction commits the move; renewed violation rolls it back to the previous epoch and domain. The protocol therefore distinguishes a predicted improvement from a verified recovery.

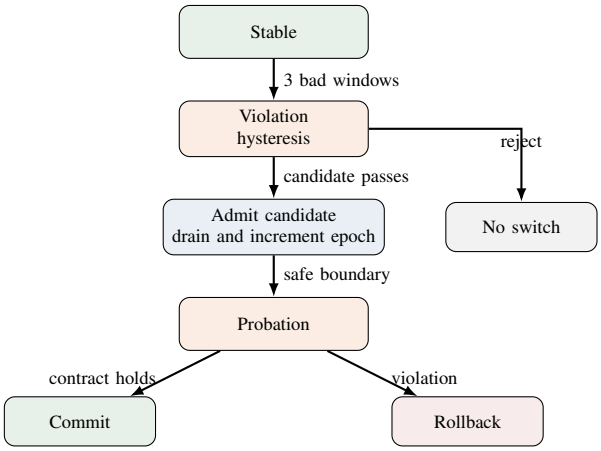


Fig. 2. Contract recovery. Prediction authorizes probation, not commitment; each terminal outcome returns the monitor to Stable.

V. IMPLEMENTATION

The implementation comprises a Python command-line controller and evidence pipeline, generated C++ ABI code, an AArch64 Linux executor, and Arduino/Zephyr sketches. The policy executes in Linux userspace. Bridge RPC carries task calls and diagnostics; the stock router remains responsible for transport. A resource probe reports compile-time capacities and explicitly marks runtime heap and stack statistics unavailable when the stock firmware cannot expose them safely.

Instrumentation is layered. Reduced mode records core timing; full mode adds structured observations; disabled mode measures the base call path. An optional kernel probe is evaluated separately and is not required for control. LED matrix updates have disabled, static, and 10/30/60-Hz refresh profiles, making the visual status display an experimentally bounded feature rather than unmeasured decoration. All records carry schema versions, run identifiers, monotonic timestamps, outcomes, payload sizes, and relevant mode metadata.

The artifact [18] includes command-line audits for each development stage. The final audit cross-checks source hashes,

TABLE II
EXPERIMENTAL SETUP

Item	Configuration
Board	Arduino UNO Q
Linux domain	Arm Cortex-A53, Debian, AArch64
RT domain	STM32U585 Cortex-M33, Zephyr
Transport	Arduino Bridge RPC / MessagePack
Task	Bounded signed 32-bit integer addition
Per-cell evidence	1000observations
Per-node deadline	20ms, pre-derived
Policy location	Linux userspace
Claim class	Empirical soft real-time

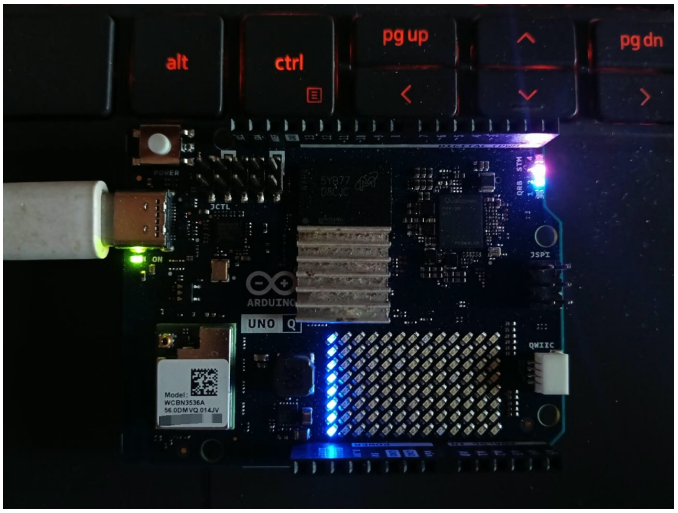


Fig. 3. Physical testbed displaying audited decision 2: an accepted Linux-to-RT transition. Matrix rows encode the Linux source, RPC boundary, and RT destination; the single illuminated column is the prototype's one QTask.

sample counts, baseline coverage, derived deadlines, negative safety scenarios, direct hardware recovery and rollback reports, graph outputs, and headline numbers.

VI. EVALUATION METHODOLOGY

A. Testbed and evidence classes

Table II summarizes the setup. Measurements were collected on a single UNO Q using its stock Linux–Zephyr Bridge. Performance campaigns fix the Linux CPU governor during a run and restore it afterward. Loaded campaigns use controlled CPU contention. Each main evaluation cell retains 1000observations with a fixed recorded seed.

We keep three evidence classes separate. *Direct measurements* are timing observations produced on the board. *Hardware scenarios* exercise the real controller and demonstrate one successful commit and one rollback. Finally, *measured-trace replay* composes sampled board traces according to a fixed application graph, placement, edge cost, and seed. Replay permits paired, repeatable comparison across seven policies, but it is not equivalent to running four complete application binaries.

TABLE III
BASELINE ROLES AND INFORMATION

Baseline	Role
Ordinary Linux	Default deployment, no RT tuning
Tuned Linux	Deployable expert scheduler configuration
MCU-first	Static preference for eligible RT nodes
Manual expert	Declared per-graph hand placement
QFabric static	Profile-driven, no online recovery
QFabric recovery	Causal monitor and guarded switch
Offline oracle	Non-causal future-trace lower bound

Application-graph deadlines scale the pre-derived per-node deadline by graph size: 60 ms for each three-node graph and 80 ms for the four-node flagship. This rule is fixed before replay and is applied uniformly across policies.

B. Applications and baselines

The four models are a four-node synthetic chain, a three-node signal pipeline, a three-node control loop, and a three-node network pipeline. Nodes differ in semantic eligibility, including Linux-only operations. The flagship scenario is the four-node chain under a Linux load shift with an 80-ms graph deadline.

We compare: (1) ordinary Linux; (2) Linux under `SCHED_FIFO`; (3) a static MCU-first rule; (4) a declared manual-expert placement; (5) QFabric's static recommendation; (6) QFabric with recovery; and (7) an offline oracle that chooses with future trace knowledge. The oracle is a lower-bound reference, not a deployable competitor. Table III makes the information advantage explicit.

C. Metrics and audit

Primary metrics are end-to-end p95 latency and deadline-miss rate. We also report detection and verification delay in complete windows, false switches, rejection coverage, decision cost, traced Python allocation peak, communication latency, profiling perturbation, kernel-probe overhead, and LED visualization overhead. All percentile calculations and figures originate from machine-readable data. The generator records the source SHA-256 in a manifest.

VII. RESULTS

A. RQ1–RQ3: detection, recovery, and safety

In the controlled hardware scenario, the sustained violation was detected one complete window after onset, with zero false switches. A feasible alternate was switched, observed, and verified after four windows. A separate injected scenario forced probation failure and produced a rollback. The negative suite exercised nine stable rejection reasons: insufficient evidence, unsatisfied contract, semantic ineligibility, destination

admission failure, MCU headroom, predicted deadline miss, higher predicted end-to-end cost, observed miss-rate failure, and active cooldown. These are bounded scenario results, not a proof that all failures are impossible.

B. RQ4: comparative effectiveness

Table IV and Fig. 4 give the central comparison. Ordinary Linux and the static QFabric recommendation have identical failing behavior because both keep all four nodes in Linux. Recovery shifts eligible work and reduces p95 from 111.30 to 70.61 ms (36.6%), reducing misses from 67.3% to 3.7%. The offline oracle places all nodes in RT and achieves 43.45 ms with no misses.

The strongest deployable baseline is tuned Linux: 23.91 ms with no deadline misses. Recovery is 2.95× its p95 and 1.63× the oracle's. The tuned result is not an anomaly to discard; it is the main boundary on our claim. If the workload can remain in Linux and the deployment can be correctly configured, scheduler tuning is both simpler and faster in this experiment. QFabric is valuable when the original placement is already deployed, its empirical contract degrades, and a tested alternate is available.

Under idle traces, the selected static placement does not improve all-Linux execution for these small tasks. For example, the synthetic chain has a 31.72-ms all-Linux p95, while MCU-first reaches 43.53 ms and the declared manual placement 56.04 ms. Boundary and RT invocation costs dominate the trivial addition kernel. This reinforces why movement is a recovery action rather than a universal optimization.

C. RQ5: overhead

Table V reports measured overhead components. One policy decision costs 170.1 μs CPU time and 177.2 μs wall time at p95 over 1,000 in-process evaluations, with a 3138-byte peak observed by `tracemalloc`. That memory value excludes the interpreter baseline and native allocator, so it is not total process memory.

Communication dominates policy cost: p95 is 8.05 ms for the Linux path and 11.42 ms for the RT path. Full profiling changes Linux p95 by -2.41% and RT p95 by $+1.39\%$ relative to disabled mode; the negative Linux delta is measurement variation, not a speedup claim. Optional kernel instrumentation adds 3.76%at p95. The worst LED visualization delta is 2.05%. These results support userspace control at the tested rate, but not arbitrary high-frequency control.

D. RQ6: programmer effort

The visible task declaration is a compact header that specifies types, effect, and task identifier once. The generated ABI and common command-line workflow remove hand-written marshaling from application code. However, the programmer must still supply semantically equivalent Linux and RT implementations, choose a defensible contract, and state any Linux-only effects. QFabric shifts effort from transport plumbing toward explicit contracts and equivalence; it does not eliminate dual-domain implementation or validation.

VIII. Discussion

The experiment exposes three practical lessons. First, cross-domain placement must include communication. The addition kernel is so small that Bridge cost dominates, making static migration unattractive under idle conditions. Second, recovery and tuning solve different problems. Tuned Linux is the right answer when it can be applied reliably; contract recovery is a fallback when observed behavior invalidates the current deployment. Third, a predicted faster domain is not sufficient. Semantic eligibility, capacity admission, quiescence, and post-switch verification are necessary to make adaptive placement operationally credible.

The LED matrix is intentionally peripheral to the novelty. It exposes coarse controller state for a physical demonstration, and its refresh cost is measured, but it does not participate in scheduling decisions. This keeps presentation quality from distorting the research claim.

A. Threats to validity and limitations

External validity. Results come from one UNO Q, one software stack, one concrete QTask, and a single board instance. Other heterogeneous boards, transports, kernels, thermal regimes, or workloads may reverse placement costs. The application study replays measured traces over models; full applications were not deployed end to end.

Construct validity. Empirical p95/p99 and miss rates do not establish worst-case execution time or hard real-time guarantees. The RT admission test is a conservative utilization gate, not a complete schedulability proof. MCU runtime heap and stack watermarks were unavailable on stock firmware and are reported as such rather than inferred.

Internal validity. Measurements share a board and may be affected by uncontrolled firmware, caching, temperature, and OS activity. Fixed seeds and paired trace replay improve repeatability but can underrepresent live interactions. The deadline is derived before replay from Stage 1 data, which reduces post-hoc bias but does not make it application-specific.

Baseline validity. `SCHED_FIFO` is a strong but not exhaustive Linux baseline; `SCHED_DEADLINE`, CPU isolation, affinity, and kernel tuning warrant future comparison. The manual baseline depends on the declared expert mapping, while the oracle has non-causal information. We report both roles explicitly.

Safety scope. Quiescent switching prevents mixed-epoch calls in the prototype, and effect annotations reject known-ineligible tasks. This is not a formal proof of implementation equivalence. Production use would require stronger state-transfer semantics, authentication, fault containment, and multi-client concurrency analysis.

IX. Artifact and Reproducibility

The public repository [18] contains source, generated code, unit tests, board sketches, raw and processed results, campaign logs, negative fixtures, SVG outputs, and stage audits. The final evaluation JSON is the source of truth for this manuscript's quantitative macros, Table IV, and Fig. 4. Running `make` in

TABLE IV
FLAGSHIP FOUR-NODE LOAD-SHIFT RESULT. L DENOTES LINUX AND R DENOTES RT; RECOVERY AND ORACLE PLACEMENTS VARY OVER THE TRACE.

Baseline	p95 (ms)	Miss rate (%)	Placement
ordinary-linux	111.30	67.3	L/L/L/L
tuned-linux	23.91	0.0	L/L/L/L
static-mcu-first	43.45	0.0	R/R/R/R
manual-expert	100.93	58.3	L/R/R/L
qfabric-static	111.30	67.3	L/L/L/L
qfabric-recovery	70.61	3.7	adaptive
offline-oracle	43.45	0.0	per observation

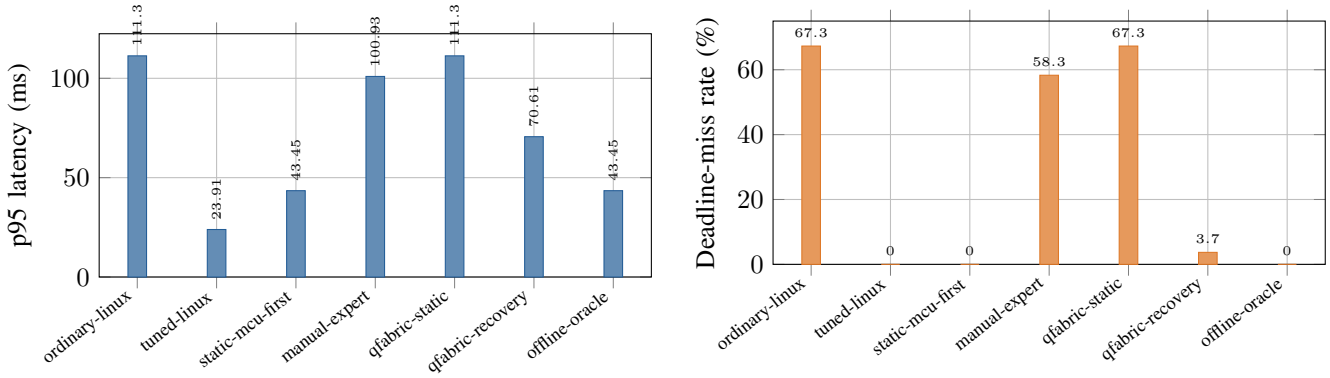


Fig. 4. Flagship end-to-end result from deterministic replay of measured board traces. The graph deadline is 80 ms.

TABLE V
MEASURED MECHANISM COSTS

Component	Result
Policy CPU time, p95	170.1 μs
Policy wall time, p95	177.2 μs
Policy traced peak	3138 bytes
Linux communication, p95	8.05 ms
RT communication, p95	11.42 ms
Full profiling delta, Linux	-2.41%
Full profiling delta, RT	$+1.39\%$
Optional kernel probe delta	3.76%
Worst LED display delta	2.05%

TABLE VI
ANSWERS TO THE RESEARCH QUESTIONS

RQ	Audited answer
RQ1	Yes, with one-window detection in the controlled scenario and no false switch.
RQ2	Yes for the tested feasible alternate; p95 improved 36.6%.
RQ3	Yes for nine enumerated negative scenarios plus hardware rollback.
RQ4	Better than ordinary Linux, but worse than tuned Linux and the oracle.
RQ5	Policy, transport, profiling, kernel, stabilization, and display costs quantified.
RQ6	Declaration and generated ABI reduce plumbing; dual implementations remain required.

the `paper` directory first regenerates those assets, records the input digest, and then builds the PDF. The included testbed photograph shows the physical board used for the reported experiments, with identifying details redacted; it can be replaced without changing the manuscript source.

For archival citation, the latest tagged repository release at manuscript build time is `v1.0.1`. The repository is prepared for deposition in a DOI-issuing archive, and its releases retain the audited Stage 11 evidence used to generate this manuscript's results.

X. CONCLUSION

QFabric demonstrates a narrow but useful form of runtime adaptation across a Linux–RTOS boundary: empirical contract violations initiate a guarded, quiescent switch whose benefit must be observed before commitment. On one UNO Q testbed, the mechanism detects controlled degradation, rejects enumerated unsafe moves, commits a feasible recovery, and rolls back a failed one. Measured-trace replay shows that recovery cuts flagship p95 by 36.6%relative to failing ordinary Linux, but remains almost 2.95$\times$ slower than tuned Linux. That result defines the contribution honestly. QFabric is not a universal scheduler or a hard-real-time proof; it is an auditable recovery layer for deployments whose original empirical timing assumptions can cease to hold. Broader tasks, boards, live application graphs, and stronger Linux baselines are the next tests of generality.

REFERENCES

[1] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE*

Transactions on Parallel and Distributed Systems, vol. 13, no. 3, pp. 260–274, 2002.

[2] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

[3] C. Lu, J. A. Stankovic, G. Tao, and S. H. Son, "Feedback control real-time scheduling: Framework, modeling, and algorithms," *Real-Time Systems*, vol. 23, pp. 85–126, 2002.

[4] T. Reinbacher, M. Függer, and J. Brauer, "Runtime verification of embedded real-time systems," *Formal Methods in System Design*, vol. 44, pp. 203–239, 2014.

[5] P. R. Nandi, A. Monot, and M. Oriol, "Stochastic contracts for runtime checking of component-based real-time systems," in *Proceedings of the 18th International ACM SIGSOFT Symposium on Component-Based Software Engineering*, 2015, pp. 138–147.

[6] Arduino, *UNO Q User Manual*, Arduino, 2026, accessed 2026-10-09. [Online]. Available: https://docs.arduino.cc/tutorials/uno-q/user-manual/

[7] ——, *Arduino RouterBridge Library*, Arduino, 2026, accessed 2026-10-09. [Online]. Available: https://github.com/arduino-libraries/Arduino_RouterBridge

[8] Linux Kernel Developers, *Remote Processor Framework*, The Linux Kernel Documentation, 2026, accessed 2026-10-09. [Online]. Available: https://docs.kernel.org/staging/remoteproc.html

[9] ——, *Remote Processor Messaging (rpmsg) Framework*, The Linux Kernel Documentation, 2024, accessed 2026-10-09. [Online]. Available: https://www.kernel.org/doc/html/v6.8/staging/rpmsg.html

[10] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003.

[11] M. A. Khatib, A. Girard, and T. Dang, "Verification and synthesis of timing contracts for embedded controllers," in *Proceedings of the 19th International Conference on Hybrid Systems: Computation and Control*, 2016.

[12] M. Camilli, A. Gargantini, and P. Scandurra, "Zone-based formal specification and timing analysis of real-time self-adaptive systems," *Science of Computer Programming*, vol. 159, pp. 28–57, 2018.

[13] B. Pourmohseni, A. Smirnov, S. Wildermann, and J. Teich, "Real-time task migration for dynamic resource management in many-core systems," in *Next Generation Real-Time Embedded Systems*, ser. OpenAccess Series in Informatics, vol. 77, 2020, pp. 5:1–5:14.

[14] J. Lelli, C. Scordino, L. Abeni, and D. Faggioli, "Deadline scheduling in the Linux kernel," *Software: Practice and Experience*, vol. 46, no. 6, pp. 821–839, 2016.

[15] M. Goraczko, C.-C. Han, J. Helander, J. Liu, B. Priyantha, and F. Zhao, "CoMOS: An operating system for heterogeneous multi-processor sensor devices," Microsoft Research, Tech. Rep. MSR-TR-2006-177, 2006.

[16] B. Rouxel, S. Altmeyer, and C. Grelck, "YASMIN: A real-time middleware for COTS heterogeneous platforms," in *Proceedings of the 22nd International Middleware Conference*, 2021, pp. 298–309.

[17] A. Yano, R. Nishimura, Y. Kobayashi, Y. Takano, T. Ishikawa-Aso, and T. Azumi, "Awkernel: RTOS bridging DAG scheduling theory and component-oriented real-time systems practice," 2026.

[18] A. Bhattacharya and A. Chakraborty, "QFabric: Contract-driven cross-domain task recovery," 2026, research artifact and replication package, Apache-2.0. [Online]. Available: https://github.com/elixpo/qfabric